

ProtoMegaBot / OmegaClaw

Current Installation, Architecture, Telegram Configuration, and Operational Reliability

Evidence snapshot: 2026-07-12, approximately 21:19 PDT

Prepared by ZeroBot for Benjamin Goertzel

Scope. This report describes the project-local OmegaClaw installation that powers @Protomegabot, its PeTTa/SWI-Prolog/Python/OpenClaw stack, its hybrid Telegram Bot API and MTPROTO path, the state observed during inspection, recurrent failures, and a prioritized remediation plan.

Security note. No bot token, API hash, Gateway token, session contents, or other credential value is reproduced. Paths, configuration key names, non-secret chat identifiers, modes, process metadata, and redacted credential-file metadata are included where operationally necessary.

Epistemic convention. “Observed” means directly inspected in source, process state, metadata, or logs. “Historical evidence” means recorded in dated project records or logs. “Inferred” identifies a reasoned diagnosis that still requires a controlled test.

Contents

1	Executive summary	3
2	Scope, evidence, and limitations	3
2.1	Evidence inspected	3
2.2	Limitations	4
3	Installation topology	4
3.1	Directory layout	4
3.2	Installed runtime versions	4
3.3	Source-control condition	4
4	Launch, supervision, and process ownership	5
4.1	Runner	5
4.2	Shell supervisor	5
4.3	Candidate systemd service	5
4.4	Observed process state	5
5	End-to-end architecture and message flow	6
5.1	Nominal inbound path	6
5.2	Model-provider path	6
5.3	Outbound path	6
5.4	Routing invariant	6
6	Telegram configuration	6
6.1	Why MTProto was added	7
6.2	Bot API capabilities retained	7
6.3	MTProto bridge design	7
6.4	Confirmed transport-mode leakage	7
6.5	Chat and response filtering	7
6.6	Attachments	8
7	Security and credential handling	8
7.1	What is handled correctly	8
7.2	Session-file exposure	8
7.3	Other boundaries	8
8	Failure chronology	8
9	Persistent issue analysis	9
9.1	Native SWI/Janus instability	9
9.2	Bridge lifecycle and orphaning	9

9.3	Health-check insufficiency	10
9.4	Context, feedback, and loop growth	10
9.5	Network and rate limits	10
9.6	Configuration drift	10
10	Operational situation and readiness assessment	10
10.1	What is established	10
10.2	What is not established	11
10.3	Readiness judgment	11
11	Prioritized remediation plan	11
11.1	Priority 0: immediate safety and correctness	11
11.2	Priority 1: lifecycle and observability	11
11.3	Priority 2: reproduce the native crash	11
11.4	Priority 3: end-to-end Telegram validation	11
11.5	Priority 4: reproducibility and deployment cleanup	12
12	Verification checklist	12
13	Safe inspection commands	12
14	Evidence index	13
15	Conclusion	13

1 Executive summary

ProtoMegaBot is not a single daemon. It is a layered local research runtime:

```
Telegram
|-- MTPProto / Telethon receive subprocess
|-- Bot API send, getFile, and accidental residual getUpdates
v
OmegaClaw Python Telegram adapter
v
PeTTa / MeTTa control loop on SWI-Prolog through Janus
v
OmegaClaw memory, skills, and prompt assembly
v
OpenClaw provider subprocess
v
local OpenClaw Gateway /v1/chat/completions
```

The installation is substantial and locally self-contained, but the current service boundary is fragile. The intended design is MTPProto receive plus Bot API send. The actual inspected configuration leaves `TG_SYNC_POLL=true`; therefore `getLastMessage()` continues to call Bot API `getUpdates` even after startup logs claim that Bot API polling is disabled. This matches July 12 logs containing Bot API polling timeouts, HTTP 429, HTTP 502, reset, and TLS timeout errors during MTPProto operation.

The most serious observed failure is native runtime instability. A July 12 run reached the OpenClaw model-processing path and then SWI-Prolog reported fatal signal 11. Earlier logs also contain status-137 exits and repeated termination/restart cycles. Moving Telethon into a subprocess reduced one Janus/GIL interaction, but it did not isolate the whole Telegram and model-processing pipeline from SWI/Janus failure.

At the process snapshot near 21:19 PDT, the supervisor loop, a SWI/PeTTa process, and a newly spawned Telethon bridge were running. A second Telethon bridge from an earlier dead parent remained reparented to PID 1. Thus the system was running, but it was not cleanly owned: one active stack coexisted with an orphaned receiver. A watchdog's latest status of "ok" is therefore insufficient evidence of end-to-end health.

The highest-priority corrections are:

1. Make receive transport selection exclusive: set `TG_SYNC_POLL=false` in MTPProto mode and guard `getLastMessage()` in code.
2. Give the supervisor explicit ownership of every Telethon child; remove `setsid` or add a robust parent-death/liveness mechanism, and reap stale bridge/session locks before restart.
3. Tighten the MTPProto session file from mode 0644 to 0600 and verify ownership before startup.
4. Replace PID-only watchdog success with an end-to-end health state: supervisor, SWI parent, exactly one bridge, recent receive heartbeat, and a bounded synthetic receive-to-reply canary.
5. Reproduce and localize the SWI/Janus signal-11 failure with minimal, recorded experiments before declaring the bot reliable.

2 Scope, evidence, and limitations

2.1 Evidence inspected

The report is grounded in:

- project records: `PROJECT.md`, `TASKS.md`, `DECISIONS.md`, `RUNBOOK.md`, and `NOTES.md`;
- launch and supervision files under `projects/omegaclaw/local/`;
- the active nested runtime source under `repos/PeTTa/repos/OmegaClaw-Core/`;
- current process listings, PID artifacts, file modes, installed package versions, and cron summaries;
- current and rotated logs under `artifacts/telegram-private-supervisor/`; and
- dated chronology in `memory/2026-07-12.md` and project records.

The active source tree contains uncommitted modifications and untracked runtime additions. This report describes what was present on disk and running, not solely an upstream release or clean Git commit.

2.2 Limitations

No credential values were opened or copied into this report. File key names and metadata were inspected with values redacted. No paid resource was used. The report did not intentionally stop, restart, or reconfigure the live bot. A scheduled watchdog restarted the runtime during inspection, so all process claims are timestamped snapshots.

No successful end-to-end bot-authored Telegram test was observed during report preparation. Consequently, MTProto connection is established, but dependable receipt, consumption, model response, and source-chat reply have not yet been established as a complete chain.

3 Installation topology

3.1 Directory layout

Component	Inspected location and role
Project notebook	/home/openclaw/research-agent/projects/omegaclaw; records, local launchers, policy, artifacts, and reports.
PeTTa runtime	repos/PeTTa; active working directory for <code>run.sh</code> <code>run.metta</code> .
Active OmegaClaw-Core	repos/PeTTa/repos/OmegaClaw-Core; Telegram adapter, OpenClaw provider, prompts, memory, skills, and extensions used by the live PeTTa run.
Inspection clone	repos/OmegaClaw-Core; separate clone retained for upstream/source comparison. It is not the active nested runtime tree.
SWI-Prolog	local/swipl-9.3.36; project-local SWI-Prolog distribution selected through <code>PATH</code> , <code>SWI_HOME_DIR</code> , and <code>LD_LIBRARY_PATH</code> .
Python environment	repos/PeTTa/.venv; Python 3.10.12, Janus binding, Telethon 1.44.0, ChromaDB, sentence-transformers, and other runtime dependencies.
Vector memory	repos/PeTTa/chroma_db; persistent local Chroma store.
Model caches	local/huggingface and local/sentence_transformers; local embedding/model cache paths exported by the runner.
Secret environment	/home/openclaw/.openclaw/omegaclaw-telegram.env; bot and Telegram application credentials, observed mode 0600.
MTProto session	/home/openclaw/.openclaw/protomegabot_mtproto.session; Telethon authorization/session database, observed mode 0644.
Runtime logs	artifacts/telegram-private-supervisor/; current and rotated stdout/stderr, bridge stderr, and historical PID artifacts.

3.2 Installed runtime versions

Observed:The active virtual environment reported Python 3.10.12 and Telethon 1.44.0. The local SWI path is named for SWI-Prolog 9.3.36. The Janus Python module imported successfully. These are mutable local facts and should be captured with exact package inventories in any reproducer.

3.3 Source-control condition

Observed:The active nested OmegaClaw-Core tree had modified files including `channels/telegram.py` and `lib_llm_ext.py`, plus untracked MTProto bridge files and tests. The PeTTa tree also contained an untracked virtual environment, Chroma database, nested repositories, and runtime files. This is acceptable for active research, but it means operational behavior cannot be reconstructed from one Git commit alone.

Risk: A crash report without a source snapshot, diff, package lock, and launch environment is not fully reproducible. A coherent tested slice should be committed on a task branch, while local credentials, sessions, caches, databases, and logs remain ignored.

4 Launch, supervision, and process ownership

4.1 Runner

The runner `local/run-omegaclaw-openclaw-telegram-private.sh` performs the following actions:

1. sources the local secret environment file;
2. changes to the PeTTa directory and creates memory/cache paths;
3. exports project-local SWI, Python, GoalChainer, Chroma, and model-cache paths;
4. configures the local OpenClaw Gateway and per-call provider sessions;
5. configures Telegram access and receive mode;
6. activates the PeTTa virtual environment; and
7. executes `timeout ... sh run.sh run.metta` with Telegram, OpenClaw provider, loop bounds, memory bounds, and local policy arguments.

Notable defaults are a 900-second Gateway HTTP timeout, a 900-second provider subprocess timeout, per-call OpenClaw sessions, `TG_SYNC_POLL=true`, and `TG_USE_MTPROTO=true`. Late heavyweight extensions are disabled by default pending a bounded loader.

4.2 Shell supervisor

The shell supervisor creates a detached bash loop. It runs the runner, records the exit status, waits five seconds, and starts it again. Its current source defaults include:

- `TG_PRIVATE_ONLY=false`;
- four explicit chat targets, including Ben’s direct chat, the scheduled-updates channel, the BotBot chat -5459676079, and another group;
- `TG_USE_MTPROTO=true`; and
- a one-day outer timeout.

Observed: The usage text still says the supervisor defaults to private/direct chat only, `TG_PRIVATE_ONLY=true`, and one direct-chat target. The executable defaults say otherwise. This is concrete documentation/configuration drift.

The supervisor’s `status` command validates only the PID recorded in its PID file. It does not verify the SWI child, bridge count, Telegram connection, queue consumption, Gateway path, or reply path.

4.3 Candidate systemd service

A service definition exists at `local/omegaclaw-telegram.service`. It sets `HOME`, loads the secret env file, starts the same runner, uses `Restart=always`, and includes partial systemd hardening. Its presence in the repository does not prove it is installed, enabled, or the active owner. The observed process tree was the shell-supervisor path.

4.4 Observed process state

At approximately 21:19 PDT the process listing showed:

- a detached supervisor bash loop started around 21:17:40;
- an SWI/PeTTa process under that run;
- a Telethon bridge child associated with the current SWI run; and
- an older Telethon bridge started around 21:12:07 with parent PID 1.

Observed: The stack had restarted and was running, but two bridge processes existed, one orphaned. The current log was updating. This supersedes the earlier 21:16 snapshot in which only the orphaned bridge was visible.

Inferred:The watchdog can move the system from “dead parent plus orphan bridge” to “new parent plus old and new bridges.” If both bridges use the same bot session and receive stream, duplicate consumers, session-database contention, dropped ownership, and ambiguous health are plausible.

5 End-to-end architecture and message flow

5.1 Nominal inbound path

1. Telethon authenticates a bot MTProto session using the Telegram application ID/hash and bot token.
2. `events.NewMessage()` invokes `_convert_message()` in `telegram_mtproto_bridge.py`.
3. The bridge converts the event into a Bot-API-shaped JSON update and writes one JSON line to stdout. It also attempts a nonblocking backup FIFO write.
4. A reader thread in `telegram_mtproto.py` parses stdout and calls the main adapter’s `_handle_updates()`.
5. The adapter extracts text and attachments, normalizes sender/chat metadata, applies chat and user policy, determines whether a group message should be context-only, and appends a tuple to `_pending_messages`.
6. `getLastMessage()` dequeues one tuple and binds `_active_chat_id` to the source chat.
7. The MeTTa loop adds prompt/history/memory context and calls the configured model provider.

5.2 Model-provider path

The OpenClaw provider in `lib_llm_ext.py` uses a child Python process to call the localhost OpenAI-compatible endpoint at `http://127.0.0.1:18789/v1/chat/completions`. The child process boundary was introduced to leave Janus with subprocess I/O and a plain string rather than a live SDK/network object.

The runner enables per-call session identifiers. This avoids accumulating OmegaClaw’s already assembled history inside one persistent Gateway conversation. Project comments record that a previous fixed Gateway user reached roughly 998k input and 272k output tokens and caused deep requests to stall or time out. HTTP and subprocess timeouts are aligned at 900 seconds for slow formalization requests.

The provider includes bounded history truncation, triage/escalation paths, retries for transient failures, diagnostics, and explicit failure responses instead of silent empty returns. These are mitigations, not proof that native SWI/Janus state remains safe while calls are pending.

5.3 Outbound path

Model output is interpreted as OmegaClaw skill calls. A `send` call reaches `telegram.send_message()`, which chooses `_active_chat_id`, then `_reply_chat_id`, then the default chat. The adapter splits text into chunks below Telegram’s limit and sends each with Bot API `sendMessage`.

The adapter also has duplicate-send suppression and an optional immediate acknowledgement for likely long requests. The latter does not depend on the model successfully emitting a `send` action.

5.4 Routing invariant

The intended invariant is:

For every dequeued inbound message m from chat c , every ordinary reply causally produced while m is active must be sent to c , regardless of later arrivals from other chats.

The current `_pending_messages` queue and `_active_chat_id` binding were added after stale latest-chat state caused wrong-chat risk. This is a critical safety property for a multi-chat bot. It should be tested under interleaving, slow provider calls, context-only bot messages, pre-acks, retries, and duplicate suppression.

6 Telegram configuration

6.1 Why MTPROTO was added

The Telegram Bot API does not provide ordinary bot-authored messages to another bot through `getUpdates` in the desired group interaction pattern. Privacy-mode and administrator changes do not remove that bot-to-bot delivery limitation. The operational objective is for ProtoMegaBot to receive other bots' messages in `ProtoBots-BotBotChats`, initially at chat target `-5459676079`.

Telethon/MTPROTO was therefore introduced for receiving. Importantly, this is a bot MTPROTO session authenticated with a bot token, not a human-account userbot login. Bot API remains the send and file-download path.

6.2 Bot API capabilities retained

The adapter retains:

- `sendMessage` for outbound replies;
- `getFile` and Bot API file URLs for attachments;
- `getUpdates` implementation for fallback receive mode;
- initial offset handling and update offsets;
- edited-message and channel-post handling;
- chat/user allowlists and private-only policy;
- source-chat reply binding; and
- acknowledgement and duplicate-send controls.

6.3 MTPROTO bridge design

`telegram_mtproto.py` launches `telegram_mtproto_bridge.py` through `setsid`. The design intent is to isolate Telethon's asyncio event loop and Python GIL activity from SWI-Prolog's embedded Janus interpreter. Bridge stdout is the primary IPC channel; a named FIFO is attempted as backup IPC.

The child emits status, message, and error JSON objects. The parent marks the adapter connected when it sees "Connected as", feeds message updates to the shared handler, and marks the adapter disconnected when the reader ends.

6.4 Confirmed transport-mode leakage

Observed:Startup in MTPROTO mode returns before the Bot API polling thread is created and logs "Bot API polling disabled." However, `start_telegram()` still sets `_sync_poll` from `TG_SYNC_POLL`; the runner defaults that variable to true. Independently, `getLastMessage()` executes `_poll_once()` whenever `_sync_poll` and `_running` are true. There is no `use_mtproto` guard in that function.

Therefore the actual flow is:

```
MTPROTO event handler -> _handle_updates(...)
                        plus
getLastMessage() -> Bot API getUpdates(...)
```

July 12 logs confirm Bot API polling errors while MTPROTO was active, including repeated timeouts, HTTP 429, HTTP 502, connection reset, and TLS handshake timeout.

Risk:This can create unnecessary request load, rate limiting, competing Bot API consumers, delayed dequeue operations, and misleading connection state. It also invalidates the startup claim that Bot API polling is disabled.

6.5 Chat and response filtering

The current supervisor source lists several chat IDs and defaults to all chat types. The adapter rejects messages outside configured chats. In groups, it can ingest messages addressed only to other bots for context while marking them `skip_response`; an explicit mention of ProtoMegaBot overrides that suppression.

Risk: Broad multi-chat operation increases the importance of exact source-chat routing and explicit response policy. The supervisor’s private-oriented name and help text should be changed or the defaults narrowed; the current mismatch is likely to mislead an operator.

6.6 Attachments

The Bot API path downloads files with explicit byte and text-character caps. Text is treated as untrusted content, PDFs can be locally converted with `pdftotext`, and large extracts are chunked.

Inferred: MTPROTO document/photo identifiers are not necessarily valid Bot API `file_id` values. The bridge currently maps MTPROTO document IDs into fields named `file_id`, while the shared attachment code later calls Bot API `getFile`. Text-only bot messages may work while MTPROTO-received attachments fail. This needs a transport-specific attachment test and likely a Telethon download path.

7 Security and credential handling

7.1 What is handled correctly

Observed: The secret env file is outside the project repository, owned by the `openclaw` user, and mode 0600. The project scripts refer to it by path and do not embed credential values. The report itself contains no credential values. The runner obtains the local Gateway token from the OpenClaw secret store only when needed and exports it to the provider process. The Telegram adapter accepts either a dedicated OmegaClaw token variable or the conventional Bot API variable, with comments warning against reusing ZeroBot’s token.

7.2 Session-file exposure

Observed: The Telethon session database was mode 0644 at inspection time. Although this host may have only one relevant local user, a Telethon session is credential-equivalent state and should not be world-readable.

Risk: A readable session database can permit account/session impersonation depending on local access and Telegram session semantics. It must be treated like a token, excluded from repositories and reports, protected at 0600, and rotated/revoked if exposure is suspected.

7.3 Other boundaries

A local filesystem/security policy is supplied to OmegaClaw. Historical evidence shows that an earlier Landlock policy blocked DNS because `/etc/resolv.conf` resolved through `/run/systemd/resolve`; the policy was amended. This illustrates that network-dependent behavior must be tested under the actual launcher and policy, not only in an unrestricted shell.

8 Failure chronology

Date	Evidence and response
2026-06-27	Initial no-reply behavior was traced first to an inactive supervisor or missing runtime token, then to DNS failure under the filesystem sandbox. The resolver target path was not permitted. Policy changes enabled a private smoke.
2026-06-27–28	Natural-language model output was not always normalized into a <code>send</code> action. Empty output and exact “No response from OpenClaw” results appeared as silence. Output normalization and diagnostics were added.
2026-06-28	Deep requests stalled. Evidence implicated a huge fixed Gateway session, misaligned short timeouts, silent backend-failure conversion, and restart behavior. Per-call sessions, aligned 900-second timeouts, visible failures, and restart-safe supervision were introduced.

Date	Evidence and response
2026-06-28	Group document use required attachment ingestion, provenance marking, bounded extraction, and chunking.
2026-07-02	Multiple chat IDs, channel posts, and source-chat replies were added.
2026-07-06	Supervisor logs show repeated restarts and status-137 runner exits. A native Telegram run was started later that day. Heavy in-loop extension imports could pin SWI; live late extensions were made opt-in.
2026-07-07	Stale chat routing and repeated <code>COMMAND_RETURN</code> feedback replay were corrected. Both could produce wrong-chat responses or prompt/history growth.
2026-07-08–10	Private-smoke preflights repeatedly found already active runtime state and broad/unsafe boundaries. Current executable defaults later drifted back to broad multi-chat operation while private-only help text remained.
2026-07-12	Bot API could not supply the desired bot-to-bot group messages, motivating MTPProto/Telethon. An initial inline Janus/Telethon approach crashed in receive integration; the bridge was moved to a subprocess.
2026-07-12	A rotated log shows successful Telethon connection and handler registration, followed by Bot API poll timeouts, HTTP 429/502, reset/TLS errors, model-processing activity, fatal signal 11 in SWI-Prolog, and termination.
2026-07-12 21:16	Process inspection briefly found no main OmegaClaw parent, but one Telethon bridge remained under PID 1.
2026-07-12 21:19	The watchdog/supervisor had started a new full stack, while the old orphan bridge remained. The resulting two-bridge state was running but not cleanly owned.

9 Persistent issue analysis

9.1 Native SWI/Janus instability

Observed:Fatal signal 11 occurred after the runtime accepted a Telegram message and entered the OpenClaw model-processing path. Earlier status-137 exits and termination cycles show a broader native-process reliability problem.

Inferred:Candidate mechanisms include unsafe interaction among SWI, Janus, Python threads, subprocess waits, callbacks, or teardown; memory pressure is also possible for status 137 but does not by itself explain signal 11. The evidence does not yet isolate one cause. The bridge subprocess removes Telethon’s event loop from the embedded interpreter, but the parent still runs Python adapter threads and provider subprocess handling through Janus.

A minimal reproducer should independently vary:

1. Bot API receive versus MTPProto receive;
2. synchronous polling on versus off;
3. no model call versus a fixed local model stub versus Gateway call;
4. no Python reader thread versus bridge reader thread;
5. clean shutdown versus parent crash/restart; and
6. one message versus interleaved multi-chat messages.

9.2 Bridge lifecycle and orphaning

Observed:`setsid` intentionally detaches the bridge from SWI’s process group. `stop_mtproto()` can terminate a known child on orderly shutdown, but a fatal SWI crash cannot execute that cleanup. The bridge can then survive under PID 1.

Inferred:The design traded signal-coupling risk for ownership loss. A correct boundary needs both isolation and liveness coupling. Options include Linux parent-death signals set in the bridge, a supervisor-owned bridge with explicit IPC, a parent heartbeat with self-termination, or a single systemd unit/cgroup that owns all descendants. Merely scanning by command line and killing all matches is unsafe unless session and project identity are checked.

9.3 Health-check insufficiency

Observed:The enabled `omegaclaw-watchdog` cron job last reported ok; nevertheless, inspection found a period with only an orphan bridge and later a period with two bridges. PID-file health is therefore not equivalent to service health.

A useful health state must assert all of the following:

- exactly one supervisor/service owner;
- exactly one SWI/PeTTa runtime for this configuration;
- exactly one MTPROTO bridge belonging to that owner;
- a recent bridge connection/receive heartbeat;
- a consuming parent reader;
- successful Gateway health within a bound;
- no restart storm; and
- periodic end-to-end receive-to-reply evidence in a designated canary chat.

9.4 Context, feedback, and loop growth

The per-call Gateway session change addresses one large source of context amplification. OmegaClaw still carries its own history, feedback, recall, prompt, and continual-loop state. Past repeated feedback records and model-generated continuation behavior have inflated prompts and caused unresponsiveness.

The current limits – history 8000, feedback 8000, eight recall items, eight episode lines, 2048 output tokens – are useful but should be measured in actual serialized prompt characters/tokens. Logs already show prompts around 20k characters for a simple channel interaction. A reliability test should track per-turn prompt size, model latency, loop count, and memory growth.

9.5 Network and rate limits

The Bot API errors on July 12 are real observations, but they are likely amplified by accidental residual polling and potentially multiple consumers. MTPROTO and Bot API both remain network-dependent. Retry policy should use bounded exponential backoff with jitter, respect rate-limit delay information where available, and separate receive health from send health.

9.6 Configuration drift

The script name and help text describe a private smoke; source defaults operate across multiple chats with private-only disabled. Secret-env variables can override shell defaults, and arguments can further override env. Without a rendered effective-config line that excludes secrets, operators cannot easily know the actual boundary.

Startup should print one redacted canonical configuration record: transport, chat IDs, private mode, allowed-user policy, session path, poll mode, timeouts, source revision, and process ownership identifier.

10 Operational situation and readiness assessment

10.1 What is established

- The project-local PeTTa/SWI/Python stack starts and loads the local embedding model and policy.
- Telethon 1.44.0 can authenticate and report connection as `@Protomegabot`.
- The MTPROTO bridge can register a new-message handler.
- The shared Telegram adapter has multi-chat allowlisting, source-chat routing, context-only group handling, attachment bounds, pre-acks, and dedupe mitigations.
- The OpenClaw provider has per-call sessions and long aligned timeouts.
- Credential values are kept outside project files; the env file is mode 0600.

10.2 What is not established

- Stable single-owner operation across crashes and restarts.
- Exclusive MTPROTO receiving without Bot API polling leakage.
- Reliable bot-authored message receipt in `ProtoBots-BotBotChats` through model processing to a correctly routed reply.
- Correct attachment behavior for MTPROTO-originated media.
- Absence of SWI/Janus signal-11 crashes under representative traffic.
- A watchdog that detects orphan, duplicate, wedged, or non-consuming bridge states.

10.3 Readiness judgment

Decision recommendation: not yet reliable for unattended broad multi-chat use. The stack is suitable for supervised diagnostic testing with bounded traffic and a clear stop condition. It should not be described as fully operational merely because Telethon connected or the watchdog returned ok.

11 Prioritized remediation plan

11.1 Priority 0: immediate safety and correctness

1. Change the MTPROTO session file to mode 0600 and add a startup check that refuses overly broad permissions.
2. Ensure MTPROTO and Bot API receive modes are mutually exclusive. In MTPROTO mode, force `_sync_poll=false`, skip `_initialize_offset()` unless a deliberate fallback is entered, and make `getLastMessage()` check the selected receive mode rather than only `_sync_poll`.
3. Stop duplicate bridge creation. Before starting, verify that no bridge for the same session is alive without the current owner. Prefer ownership by a single supervisor/cgroup rather than ad hoc process matching.
4. Reconcile the supervisor name/help text with effective defaults. For the BotBot test, name it as multi-chat and print the redacted effective configuration.

11.2 Priority 1: lifecycle and observability

1. Replace bridge detachment without liveness coupling. Recommended design: the top-level service owns both SWI and bridge; the bridge exits on parent heartbeat loss or parent-death signal; restart operates at the service boundary.
2. Add structured JSON lifecycle events with run ID, parent PID, bridge PID, transport, session basename, source revision, and monotonic timestamps. Never log token/hash values.
3. Replace the PID-only watchdog with process-tree and heartbeat checks. Alert on zero or more than one bridge, stale receive heartbeat, restart storm, signal 11, and no consumer.
4. Preserve crash logs and core/backtrace evidence with bounded retention. Capture exit reason separately for timeout, signal, OOM kill, operator stop, and normal exit.

11.3 Priority 2: reproduce the native crash

Create a recorded experiment matrix with pinned source and package inventories. Begin with a no-model fixed-message handler, then add one subsystem at a time. Use the narrowest failing case to determine whether the signal-11 trigger is receive polling, Janus callback/threading, provider subprocess interaction, or teardown.

Success criterion: at least several hundred controlled inbound events across restart and interleaving cases with zero native crashes, no orphan children, no duplicate replies, and stable memory use. The exact threshold should be chosen before the run, not after seeing results.

11.4 Priority 3: end-to-end Telegram validation

Run a supervised test in `-5459676079`:

1. record a unique canary ID and process/run IDs;
2. have ZeroBot or another bot send one explicitly addressed text message;
3. verify one Telethon event, one adapter queue insertion, one dequeue with the same chat ID, one model request, and one Bot API reply to that same chat;
4. verify no Bot API `getUpdates` call occurred;
5. verify no duplicate response and no response in another configured chat;
6. repeat with an unaddressed bot message and confirm context ingestion without reply;
7. restart once and confirm the old bridge exits and exactly one new bridge appears; and
8. separately test an MTPProto-originated attachment or explicitly disable such attachments until implemented.

11.5 Priority 4: reproducibility and deployment cleanup

Commit the coherent adapter/bridge/test slice to a task-specific branch after tests pass. Record exact PeTTa and OmegaClaw revisions, Python lock/inventory, SWI version, redacted config schema, and launch command. Select one production supervisor – preferably systemd or another single cgroup owner – and retire competing watchdog/shell ownership paths after a controlled migration.

12 Verification checklist

The following gates should all pass before calling the stack stable:

Gate	Current	Required evidence
Secret env mode 0600	Pass	<code>stat</code> metadata, no values logged.
MTPProto session mode 0600	Fail	Permission check before launch.
Exactly one service owner	Unclear	Process tree and service manager state.
Exactly one bridge	Fail at snapshot	Owner-linked PID/process count.
No Bot API receive calls in MTPProto mode	Fail	Instrumented transport test.
Bot-authored canary received	Not proven	Unique message ID in bridge and adapter logs.
Correct source-chat reply	Not proven	Matching inbound/outbound chat and canary ID.
No reply to context-only bot message	Not proven	Addressing-policy test.
Clean restart and bridge reap	Fail at snapshot	Before/after process tree.
No signal 11 under stress gate	Fail historically	Reproducible recorded run.
Attachment behavior under MTPProto	Not proven	Text/PDF/photo cases or explicit disable.
Watchdog detects duplicates/orphans	Fail by design	Injected health-failure tests.

13 Safe inspection commands

These commands expose process and metadata state but not credential values. Run them from `projects/omegaclaw`.

```
# Effective process topology
ps -eo pid,ppid,lstart,etimes,%cpu,%mem,stat,cmd \
| grep -E 'telegram_mtproto_bridge|PeTTa|OmegaClaw|swipl' \
| grep -v grep

# Credential-file metadata only; do not print contents
stat -c '%n mode=%a owner=%U group=%G size=%s' \
```

```

/home/openclaw/.openclaw/omegaclaw-telegram.env \
/home/openclaw/.openclaw/protomegabot_mtproto.session

# Recent lifecycle/failure signatures
rg -n -i 'Connected as|reader loop ended|signal 11|segv|status 137|\
429|502|timed out|Terminated' \
  artifacts/telegram-private-supervisor/*.log* | tail -n 200

# Source evidence for receive-mode selection
rg -n 'TG_USE_MTPROTO|TG_SYNC_POLL|_sync_poll|_poll_once|\
getLastMessage|start_mtproto' \
  repos/PeTTa/repos/OmegaClaw-Core/channels/telegram*.py \
  local/run-omegaclaw-openclaw-telegram-private.sh

```

14 Evidence index

Source	Main evidence
local/run-omegaclaw-openclaw-telegram-private.sh, lines 19–43, 70–109	Runtime paths, Gateway/session/timeouts, Telegram defaults, loop and memory bounds.
local/omegaclaw-telegram-private-supervisor.sh, lines 10–17, 28–53, 56–147	Broad executable defaults versus private help text; PID-only status; restart loop and process-group stop behavior.
local/omegaclaw-telegram.service, lines 1–35	Candidate systemd ownership, restart, environment, and hardening.
channels/telegram.py, lines 107–160, 622–769, 777–894	Queue/source-chat binding, access controls, shared update handling, polling, MTPROTO selection, and Bot API sending.
channels/telegram_mtproto.py, lines 1–164	Bridge subprocess, stdout reader, env loading, setsid , and orderly cleanup path.
channels/telegram_mtproto_bridge.py, lines 1–142	Telethon client, event conversion, JSON output, bot login, and event handler.
lib_llm_ext.py, OpenClawProvider section	Per-call Gateway sessions, subprocess call, history bounds, retries, escalation, and explicit failure handling.
Rotated supervisor log from 2026-07-12 21:06 PDT (under artifacts/telegram-private-supervisor/)	Successful MTPROTO startup, residual Bot API errors, user report of no answer, provider call, fatal signal 11, and termination.
Current process snapshot near 2026-07-12 21:19 PDT	Supervisor and SWI active, current bridge active, older orphan bridge under PID 1.
Redacted file metadata near 2026-07-12 21:17 PDT	Secret env mode 0600; MTPROTO session database mode 0644.
PROJECT.md, RUNBOOK.md, TASKS.md, DECISIONS.md, NOTES.md	Installation provenance, operational procedures, architecture decisions, known issues, and staged testing history.
memory/2026-07-12.md	Dated operational chronology and recent Telegram/MTPROTO work, cross-checked against code and logs.

15 Conclusion

The MTPROTO direction addresses a real Telegram delivery limitation, and the subprocess bridge is a sensible response to an observed inline Janus crash. The present implementation is nevertheless transitional. It combines MTPROTO receive with unintended Bot API polling, detaches bridge lifetime from parent lifetime, leaves a credential-equivalent

session file too broadly readable, and still exhibits fatal native crashes. The result can connect successfully yet fail as a bot service.

The shortest path to confidence is not another broad restart. It is a narrow transport-exclusivity patch, strict process ownership, permission repair, and a recorded end-to-end canary followed by a minimal crash-isolation matrix. Once those gates pass, the architecture can be simplified around one supervisor and one auditable receive path.

End of report. No secret values are included.